

État des aéronefs

Javions – étape 7

1. Introduction

Le but de cette étape est d'écrire les classes permettant de suivre l'évolution de l'état de tous les aéronefs assez proches de la radio pour que cette dernière reçoive les messages qu'ils envoient. Ce sont ces aéronefs qui seront visibles sur la carte une fois le projet terminé.

2. Concepts

2.1. Patron *Observer* et propriétés JavaFX

Les deux classes à réaliser pour cette étape utilisent ce que JavaFX appelle des **propriétés** (*properties*), qui sont des cellules – donc des objets contenant une seule valeur – observables au sens du patron *Observer*. Pour comprendre leur fonctionnement, il est donc nécessaire de connaître :

1. le patron *Observer*, décrit dans [la §1](#) du [second cours consacré aux patrons de conception](#), et
2. les propriétés JavaFX, décrites dans [la §4](#) – jusqu'à et y compris la §4.3 – du [cours consacré à JavaFX](#).

Si vous n'avez pas encore lu ces parties du cours, faites-le avant de continuer.

3. Mise en œuvre Java

Cette étape est la première de la seconde partie du projet, durant laquelle vous serez plus libres et moins guidés que durant la première.

En particulier, vous avez maintenant le droit de modifier ou augmenter l'interface publique des classes et interfaces proposées, pour peu bien entendu que vous ayez une bonne raison de le faire. Pour faciliter la correction, nous vous demandons néanmoins de respecter les noms de packages et de classes, interfaces, enregistrements et types énumérés donnés.

D'autre part, nous nous attendons à ce que vous lisiez et compreniez la documentation des parties de la bibliothèque Java que vous devez utiliser.

Les deux classes décrites ci-dessous font partie du sous-paquetage `gui`, destiné à contenir la totalité du code lié à l'interface graphique – *gui* étant l'acronyme de *graphical*

user interface.

3.1. Installation de JavaFX

Cette étape est la première utilisant la bibliothèque JavaFX, et avant de commencer à la rédiger il vous faut donc installer JavaFX en suivant les indications de [notre guide](#).

3.2. Classe `ObservableAircraftState`

La classe `ObservableAircraftState` du sous-paquetage `gui`, publique et finale, représente l'état d'un aéronef. Cet état a la caractéristique d'être observable au sens du patron de conception *Observer*.

`ObservableAircraftState` implémente l'interface `AircraftStateSetter`, et une de ses instance peut donc être associée à un accumulateur d'état de type `AircraftStateAccumulator`, afin d'être mis à jour au fur et à mesure que les messages de l'aéronef sont reçus.

Le constructeur de `ObservableAircraftState` prend en arguments l'adresse OACI de l'aéronef dont l'état est destiné à être représenté par l'instance à créer, ainsi que les caractéristiques fixes de cet aéronef, provenant de la base de données micronics. Bien entendu, ces dernières peuvent ne pas exister, la base de données n'étant pas exhaustive.

En plus de ce constructeur, `ObservableAircraftState` offre des méthodes d'accès permettant d'obtenir les attributs passés au constructeur, ainsi qu'un certain nombre de propriétés JavaFX, qui sont :

- `lastMessageTimeStampNs`, qui contient l'horodatage du dernier message reçu de l'aéronef, en nanosecondes,
- `category`, qui contient la catégorie de l'aéronef,
- `callSign`, qui contient l'indicatif de l'aéronef,
- `position`, qui contient la position de l'aéronef à la surface de la Terre (longitude et latitude, en radians),
- `trajectory`, qui contient la trajectoire de l'aéronef (voir plus bas),
- `altitude`, qui contient l'altitude de l'aéronef, en mètres,
- `velocity`, qui contient la vitesse de l'aéronef, en mètres par seconde,
- `trackOrHeading`, qui contient la route ou le cap de l'aéronef, en radians.

La trajectoire est une liste, observable mais non modifiable, des positions dans l'espace que l'aéronef a occupées depuis le premier message reçu. Chaque élément de la trajectoire est une paire constituée d'une position à la surface de la Terre (longitude et latitude) ainsi qu'une altitude. Un enregistrement public, nommé p. ex. `AirbornePos`, imbriqué dans `ObservableAircraftState`, est utilisé pour représenter ces positions dans l'espace.

`ObservableAircraftState` suit, en grande partie, les conventions utilisées pour ce

que l'on nomme parfois les *beans* JavaFX, concept décrit à [la §4.3 du cours sur JavaFX](#). Cela implique que, pour chacune des propriétés ci-dessus, cette classe offre trois méthodes publiques :

- une méthode d'accès à la propriété *en lecture seule* (voir les conseils de programmation plus bas), dont le nom est celui de la propriété suivi du suffixe `property`,
- une méthode d'accès à la *valeur* contenue dans la propriété, dont le nom est celui de la propriété précédé du préfixe `get`,
- une méthode de modification de la valeur contenue dans la propriété, dont le nom est celui de la propriété précédé du préfixe `set`.

La seule exception à cette règle est la propriété contenant la trajectoire de l'aéronef, qui n'est pas modifiable de l'extérieur et pour laquelle il n'existe donc pas de méthode de modification.

Par exemple, les trois méthodes correspondant à la propriété `category` – qui contient la catégorie de l'aéronef – sont :

- `ReadOnlyIntegerProperty categoryProperty()`,
- `int getCategory()`, et
- `void setCategory(int category)`.

Bien entendu, la dernière d'entre elles constitue la redéfinition de la méthode du même nom de l'interface `AircraftStateSetter`.

Notez que toutes ces méthodes ne seront pas forcément utilisées dans la suite du projet, mais il est bien de les définir néanmoins, dans un souci de cohérence.

3.2.1. Conseils de programmation

1. Propriétés

Les propriétés stockées dans les attributs de `ObservableAircraftState` doivent bien entendu être modifiables, faute de quoi les méthodes de modification (`set...`) ne pourraient pas changer leur contenu. Par contre, depuis l'extérieur de la classe, le contenu de ces propriétés ne devrait être modifiable *que* à travers les méthodes de modification, et pas directement. Pour garantir cela, les propriétés doivent être exportées en lecture seule, ce qui se fait très facilement en utilisant le bon type.

Par exemple, pour la propriété `category`, l'attribut (privé) doit avoir le type [IntegerProperty](#) (propriété entière modifiable), mais le type de retour de la méthode `categoryProperty` doit être [ReadOnlyIntegerProperty](#) (propriété entière en lecture seule). Sachant que le premier est un sous-type du second, la méthode `categoryProperty` peut simplement retourner l'attribut ! Bien entendu, rien n'empêche un utilisateur de cette méthode de faire un

transtypage afin de pouvoir modifier la propriété, mais cela n'est pas grave : notre but ici est de nous protéger des erreurs involontaires, et pas des utilisateurs mal intentionnés.

2. Collections observables

La technique mentionnée ci-dessus et permettant d'exporter des versions en lecture seule des propriétés ne fonctionne malheureusement pas pour les collections observables, dont il n'existe pas de version en lecture seule. Une autre stratégie, basée sur les vues non modifiables, doit donc être utilisées.

Cette stratégie consiste à avoir *deux* attributs pour chaque collection observable : le premier contenant la collection observable elle-même, bien entendu modifiable, et le second contenant une vue observable mais non modifiable sur la collection. Les méthodes d'accès retournent systématiquement la vue non modifiable, rendant ainsi impossible toute modification du contenu de la collection depuis l'extérieur.

Ainsi, deux attributs privés existent pour représenter la trajectoire de l'aéronef. Les deux sont de type `ObservableList<AirbornePos>`, mais le premier contient une liste observable et modifiable – obtenue p. ex. au moyen de la méthode `observableArrayList` –, tandis que le second contient une vue non modifiable mais observable – obtenue au moyen de `unmodifiableObservableList` – sur cette liste. La méthode d'accès retourne le second de ces attributs.

3. Calcul de la trajectoire

Chaque élément de la trajectoire est une paire (position, altitude). Or ces deux informations sont communiquées à `ObservableAircraftState` au moyen de deux appels de méthode successifs – à `setPosition` et à `setAltitude` – effectués dans un ordre a priori quelconque. De plus, l'appel à `setPosition` peut manquer dans le cas où la position de l'aéronef ne peut pas être déterminée.

Il n'est donc pas évident de savoir quand mettre à jour la trajectoire. La solution que nous vous proposons d'utiliser consiste à procéder ainsi chaque fois que la position ou l'altitude change :

- si la position est différente de la dernière de la trajectoire – ou si la trajectoire est vide – alors la paire (position courante, altitude courante) est ajoutée à la trajectoire,
- sinon, si l'horodatage du message courant est identique à celui du message qui a provoqué le dernier ajout à la trajectoire, alors le dernier élément de la trajectoire est remplacé par la paire (position courante, altitude courante).

Dans les autres cas, la trajectoire ne change pas.

3.3. Classe `AircraftStateManager`

La classe `AircraftStateManager` du sous-paquetage `gui`, publique et finale, a pour but de garder à jour les états d'un ensemble d'aéronefs en fonction des messages reçus d'eux. Dans la version finale du programme, une de ses instances gèrera les états de la totalité des aéronefs visibles sur la carte.

Pour ce faire, `AircraftStateManager` possède deux attributs : une table associant un accumulateur d'état d'aéronef à l'adresse OACI de tout aéronef dont un message a été reçu récemment, et un ensemble (observable) des états des aéronefs dont la position est connue.

À chaque accumulateur de la table est attaché un état modifiable et observable d'aéronef, c.-à-d. une instance de `ObservableAircraftState`. L'ensemble d'états ne contient quant à lui rien d'autre qu'un sous-ensemble de ces états, à savoir ceux pour lesquels la position est connue.

La table n'est pas visible depuis l'extérieur de la classe, mais l'ensemble l'est dans une certaine mesure. En effet, une méthode publique, décrite plus bas, permet d'obtenir une vue non modifiable sur lui.

La raison pour laquelle l'ensemble ne contient que les aéronefs dont la position est connue est que cela garantit d'une part qu'il est possible de les placer sur la carte, et d'autre part qu'au moins *deux* messages ADS-B ont été reçu de chacun d'entre eux. Cela diminue grandement la probabilité d'apparition d'aéronefs « fantômes » dus à des messages corrompus dont l'adresse OACI est invalide, mais le CRC valide.

`AircraftStateManager` possède un unique constructeur public, qui prend en argument la base de données contenant les caractéristiques fixes des aéronefs. Elle offre de plus trois méthodes publiques, qui sont :

- une méthode, nommée p. ex. `states`, retournant l'ensemble observable, mais non modifiable, des états observables des aéronefs dont la position est connue,
- une méthode, nommée p. ex. `updateWithMessage`, prenant en argument un message et l'utilisant pour mettre à jour l'état de l'aéronef qui l'a envoyé – créant cet état lorsque le message est le premier reçu de cet aéronef,
- une méthode, nommée p. ex. `purge`, supprimant de l'ensemble des états observables tous ceux correspondant à des aéronefs dont aucun message n'a été reçu dans la minute précédant la réception du dernier message passé à `updateWithMessage`.

3.4. Tests

À partir de cette étape, aucun fichier de vérification de signatures ne vous est fourni, étant donné que la signature des différentes méthodes n'est plus spécifiée en détail. Pour la même raison, aucun test unitaire ne sera plus fourni à l'avenir, à vous d'écrire les vôtres. Notez que cela est fortement recommandé en général.

Afin de tester cette étape et les suivantes, nous mettons à votre disposition [une archive](#)

Zip contenant un fichier nommé messages_20230318_0915.bin. Comme son nom l'indique, il contient des messages obtenus le 18 mars 2023 à partir de 9h15. Il y en a 308000 au total, reçus sur une période d'un peu plus d'une heure.

Pour économiser de la place, le fichier est un fichier binaire, constitué d'une séquence de paires composée chacune d'un horodatage et d'un message. L'horodatage est une valeur de type long occupant donc 8 octets, tandis que le message en occupe 14.

La lecture des horodatages et messages du fichier se fait assez facilement au moyen d'une instance de `DataInputStream`, un flot d'entrée d'octets offrant également des méthodes pour lire des valeurs de types primitifs autres que les octets. En utilisant ce type de flot, il est possible d'écrire un programme affichant les messages du fichier et leur horodatage ainsi :

```
try (DataInputStream s = new DataInputStream(
    new BufferedInputStream(
        new FileInputStream("messages_20230318_0915.bin")))) {
    byte[] bytes = new byte[RawMessage.LENGTH];
    while (true) {
        long timeStampNs = s.readLong();
        int bytesRead = s.readNBytes(bytes, 0, bytes.length);
        assert bytesRead == RawMessage.LENGTH;
        ByteString message = new ByteString(bytes);
        System.out.printf("%13d: %s\n", timeStampNs, message);
    }
} catch (EOFException e) { /* nothing to do */ }
```

En exécutant ce programme, l'horodatage et le contenu des 308000 messages devraient s'afficher à l'écran, les trois premières lignes étant :

```
13081000: 8D4D24802258A534DF38208E8EED
36079000: 8D3C4DC8998CA882E0A409307559
104283800: 8D40627999907492F858094A0D56
```

En vous inspirant du programme ci-dessus, vous pouvez en écrire un qui :

- crée une instance de `AircraftStateManager`,
- fournisse les messages du fichier à sa méthode `updateWithMessage`, les uns après les autres,
- après chaque message reçu, imprime une table avec les principales informations des aéronefs.

La table pourrait ressembler à ceci, les flèches sur le côté droit donnant une idée de la direction de déplacement de l'aéronef :

OACI	Indicatif Immat.	Modèle	Longitude	Latitude	Alt.	Vit.
------	------------------	--------	-----------	----------	------	------

344645		EC-MEA	AIRBUS A-320	6,01716	46,17673	1638	414 ✓
3C4DC8	DLH6K	D-ACNH	BOMBARDIER Regio...	6,05822	46,21307	1151	317 ←
3C6424		D-AIAD	AIRBUS A-321	5,38424	46,08902	10660	748 ✓
3C6443		D-AIBC	AIRBUS A-319	4,78893	45,35490	10363	846 ↗
407739		G-TUMH	BOEING 737 MAX 8	6,22349	46,31589	975	322 ✓
4402F2		OE-IZE	AIRBUS A-320	6,87825	46,55768	6294	767 ↗
4B1883		HB-JHJ	AIRBUS A-330-300	6,35760	46,40634	1890	342 ✓
4CAC62		EI-AZA	BOEING 737-800	6,74567	46,10762	10973	NaN ↑

Pour plus de réalisme, votre programme peut ne fournir un message à l'instance de `AircraftStateManager` que lorsque le temps correspondant à son horodatage s'est effectivement écoulé depuis le début du programme. Pour ce faire, vous pouvez utiliser les méthodes `nanoTime` – pour calculer le nombre de nanosecondes écoulées depuis le début du programme – et `sleep` – pour attendre un certain nombre de millisecondes.

Afin que l'évolution du contenu de la table soit plus facile à suivre, vous pouvez trier les aéronefs en fonction de leur adresse OACI. Pour ce faire, placez les états du `AircraftStateManager` dans une liste modifiable, puis triez son contenu au moyen de la méthode `sort`. Cette méthode attend un *comparateur* en argument, c.-à-d. une instance d'une classe implémentant l'interface `Comparator`. Le but d'un comparateur est de comparer deux valeurs d'un type donné, ici des états d'aéronefs. La classe suivante définit un comparateur les comparant par adresse OACI, et une de ses instances peut donc être passé à `sort` :

```
private static class AddressComparator
    implements Comparator<ObservableAircraftState> {
    @Override
    public int compare(ObservableAircraftState o1,
                      ObservableAircraftState o2) {
        String s1 = o1.address().string();
        String s2 = o2.address().string();
        return s1.compareTo(s2);
    }
}
```

Si vous êtes particulièrement motivé, vous pouvez faire en sorte que votre programme efface le contenu de l'écran avant chaque affichage de la table, au moyen de *séquences d'échappement ANSI*. Par exemple, l'extrait de programme suivant efface la totalité de l'écran :

```
String CSI = "\u001B[";
String CLEAR_SCREEN = CSI + "2J";
System.out.print(CLEAR_SCREEN);
```

Notez toutefois que les séquences d'échappement ANSI ne sont pas reconnues par la console d'IntelliJ, et il vous faudra donc exécuter votre programme depuis un émulateur

de terminal. Vous pouvez soit utiliser celui de votre système d'exploitation – p. ex. Terminal sur macOS – soit celui intégré à IntelliJ.

Pour utiliser ce dernier, ouvrez le menu *View* puis *Tool Windows* et enfin *Terminal*. Dans la fenêtre qui s'ouvre, lancez votre programme en entrant la commande suivante, après avoir remplacé `${JFX_PATH?}` par le chemin d'accès au répertoire `lib` de JavaFX, et `${MAIN_CLASS?}` par le nom complet de la classe principale de votre programme :

```
java --enable-preview \  
  -cp out/production/Javions/ \  
  --module-path ${JFX_PATH?} \  
  --add-modules javafx.controls \  
  ${MAIN_CLASS}
```

La vidéo ci-dessous montre ce à quoi pourraient ressembler les 15 premières secondes de l'exécution de votre programme dans l'émulateur de terminal d'IntelliJ :



4. Résumé

Pour cette étape, vous devez :

- écrire les classes `ObservableAircraftState` et `AircraftStateManager` selon les indications données ci-dessus,
- tester votre code,
- documenter la totalité des entités publiques que vous avez définies.

Aucun rendu n'est à faire pour cette étape avant le rendu final. N'oubliez pas de faire régulièrement des copies de sauvegarde de votre travail en suivant [nos indications à ce sujet](#).